(Task 1 of 10) In this tutorial, we will learn even more about definitions. 0

Pseudo | 🎲

(Task 2 of 10) What is the result of running this program?

Pseudo [Run ▶] Scala 3

```
let x = 12 / 0
print(3)
```

```
val x = 12 / 0
println(3)
```

⚠️ JavaScript behaves differently.

`error` 2

Please briefly explain why you think the answer is `error`. 3

because in smol, x = 12 / 0 is declared but not used, it would still run 4

You predicted the output correctly 🎉 🎉 🎉 5

When you define a variable (in this case, `x`), you have to bind it to a value, no matter whether or not you need the value of that variable later in the program. The program errors when it tries to evaluate `12 / 0`.

Click here to run this program in the Stacker.

Pseudo | 🎲

(Task 3 of 10) What is the result of running this program?

Pseudo [Run ▶] Python

```
fun f(x):
  return 3
end
print(f(7 / 0))
```

```
def f(x):
    return 3
print(f(7 / 0))
```

⚠️ JavaScript behaves differently.

`error` 7

You predicted the output correctly 🎉 🎉 🎉 8

Function calls bind their formal parameters (in this case, there is one formal parameter, `x`) to the values of actual parameters (in this case, there is one actual parameter, `7 / 0`). The program errors when it tries to evaluate `7 / 0`.

Click here to run this program in the Stacker.

Pseudo |

(Task 4 of 10) What is the result of running this program?

Pseudo [Run ▶] Python

```
let x = y + 1
let y = 2
print(x)
print(y)
```

```
x = y + 1
y = 2
print(x)
print(y)
```

`error` 10

Please briefly explain why you think the answer is `error`. 11

y is not declared on the top 12

You predicted the output correctly 🎉 🎉 🎉 13

The first definition tries to bind `x` to the value of `y + 1`. To evaluate `y + 1`, we need the value of `y`. But `y` is not bound to a value at that moment.

Click here to run this program in the Stacker.

(Task 5 of 10) In what order are definitions and expressions evaluated? 14

top to bottom 15

(Task 6 of 10) - Variables are bound to values. Specifically, every variable definition evaluates the expression immediately and binds the variable to the value, even if the variable is not used later in the program; every function call evaluates the actual parameters immediately and binds the values to formal parameters, even if the formal parameter is not used in the function. 16

- Every block evaluates its definitions and expressions in reading order (i.e., top-to-bottom and left-to-right).

Any feedback regarding these statements? Feel free to skip this question. 17

(You skipped the question.) 18

(Task 7 of 10) Please scroll back and select 1-3 programs that together make these points. 19

> - Variables are bound to values. Specifically, every variable definition evaluates the expression immediately and binds the variable to the value, even if the variable is not used later in the program; every function call evaluates the actual parameters immediately and binds the values to formal parameters, even if the formal parameter is not used in the function.

You don't need to select all such programs.

(*You selected 2 programs*) 20

Okay. How do these programs (1,9) support the point? 21

they bind the the value although it was called or not called 22

(Task 8 of 10) Please scroll back and select 1-3 programs that together make these points. 23

> - Every block evaluates in top-to-bottom, left-to-right order.

You don't need to select all such programs.

(*You selected 2 programs*) 24

Okay. How do these programs (6,9) support the point? 25

because it ran at the top, it was able to recall the stored variable which why it give error 26

(Task 9 of 10) Here is a program that confused many students 27

Pseudo [Run ▶] Lispy

```
fun addy(x):        (deffun (addy x)
  return x + y        (+ x y))
end                 (defvar s (addy 1))
let s = addy(1)     (defvar y 2)
let y = 2
print(s)            s
```

Please

1. Run this program in the stacker by clicking one of the [Run ▶] buttons above;
2. The stacker would show how this program produces its result(s);

3. Keep clicking ⏭ Next until you reach a configuration that you find particularly helpful;
4. Click 🔗 Share This Configuration to get a link to your configuration;
5. Submit your link below;

28

https://smol-tutor.xyz/stacker/?syntax=Pseudo&randomSeed=smol-tutor&hole=%E2%80%A2&nNext=2&program=%0A%28deffun+%28addy+x%29%0A++%28%2B+›

29

(Task 10 of 10) Please write a couple of sentences to explain how your configuration explains the result(s) of the program.

30

so it bind x = 1 but y wasn't called / reach so y was undeclare when it ran

31

Let's review what we have learned in this tutorial.

- Variables are bound to values. Specifically, every variable definition evaluates the expression immediately and binds the variable to the value, even if the variable is not used later in the program; every function call evaluates the actual parameters immediately and binds the values to formal parameters, even if the formal parameter is not used in the function.
- Every block evaluates its definitions and expressions in reading order (i.e., top-to-bottom and left-to-right).

You have finished this tutorial 🎉 🎉 🎉

Please print the finished tutorial to a PDF file so you can review the content in the future. **Your instructor (if any) might require you to submit the PDF.**

Start time: 1776741893770